Software Requirements Specification for Software Engineering: subtitle describing software

Team 22, Ghoti
Hemal Rajput
Gurmanjot Minhas
James Araiche
Lily Porter
Sarah Dorfman

October 19, 2026

Contents

1 Reference Material **iv**
- 1.1 Table of Units . . . iv
- 1.2 Table of Symbols . . . iv
- 1.3 Abbreviations and Acronyms . . . v
- 1.4 Mathematical Notation . . . v

2 Introduction **2**
- 2.1 Purpose of Document . . . 2
- 2.2 Scope of Requirements . . . 2
- 2.3 Characteristics of Intended Reader . . . 3
- 2.4 Organization of Document . . . 3

3 General System Description **4**
- 3.1 System Context . . . 4
- 3.2 User Characteristics . . . 5
- 3.3 System Constraints . . . 5

4 Specific System Description **6**
- 4.0.1 Scope Decisions . . . 6
- 4.0.2 Assumptions . . . 6
- 4.0.3 Theoretical Models . . . 7
- 4.0.4 General Definitions . . . 7
- 4.0.5 Data Definitions . . . 8

5 Requirements **8**
- 5.1 Functional Requirements . . . 8
  - 5.1.1 Audio Capture Requirements . . . 8
  - 5.1.2 Session Management Requirements . . . 8
  - 5.1.3 Speech Analysis Requirements . . . 9
  - 5.1.4 Audio Capture Requirements . . . 9
  - 5.1.5 Session Management Requirements . . . 10
  - 5.1.6 Speech Analysis Requirements . . . 10
- 5.2 Nonfunctional Requirements . . . 11

6 Likely Changes **12**

7 Unlikely Changes **13**

8 Traceability Matrices and Graphs **13**

9 Development Plan **14**

10 Values of Auxiliary Constants **17**

Revision History

Date	Version	Notes
Date 1	1.0	Notes
Date 2	1.1	Notes

[This template is intended for use by CAS 741. For CAS 741 the template should be used exactly as given, except the Reflection Appendix can be deleted. For the capstone course it is a source of ideas, but shouldn't be followed exactly. The exception is the reflection appendix. All capstone SRS documents should have a reflection appendix. —TPLT]

1 Reference Material

This section records information for easy reference.

1.1 Table of Units

Throughout this document SI (Système International d'Unités) is employed as the unit system. In addition to the basic units, several derived units are used as described below. For each unit, the symbol is given followed by a description of the unit and the SI name.

symbol	unit	SI
m	length	metre
kg	mass	kilogram
s	time	second
°C	temperature	centigrade
J	energy	joule
W	power	watt ($W = J\,s^{-1}$)

[Only include the units that your SRS actually uses. —TPLT]

[Derived units, like newtons, pascal, etc, should show their derivation (the units they are derived from) if their constituent units are in the table of units (that is, if the units they are derived from are used in the document). For instance, the derivation of pascals as $Pa = N\,m^{-2}$ is shown if newtons and m are both in the table. The derivations of newtons would not be shown if kg and s are not both in the table. —TPLT]

[The symbol for units named after people use capital letters, but the name of the unit itself uses lower case. For instance, pascals use the symbol Pa, watts use the symbol W, teslas use the symbol T, newtons use the symbol N, etc. The one exception to this is degree Celsius. Details on writing metric units can be found on the NIST web-page. —TPLT]

[The table should include the basic units and derived units that are used, with basic and derived defined as by SI. Units that aren't named, aren't normally in the table of units. For instance, the units m/s^2 for acceleration won't normally appear in the table. —TPLT]

1.2 Table of Symbols

The table that follows summarizes the symbols used in this document along with their units. The choice of symbols was made to be consistent with the heat transfer literature and with existing documentation for solar water heating systems. The symbols are listed in alphabetical order.

symbol	unit	description
A_C	m^2	coil surface area

A_{in}	m^2	surface area over which heat is transferred in

[Use your problems actual symbols. The si package is a good idea to use for units. —TPLT]

1.3 Abbreviations and Acronyms

symbol	description
A	Assumption
DD	Data Definition
GD	General Definition
GS	Goal Statement
IM	Instance Model
LC	Likely Change
PS	Physical System Description
R	Requirement
SRS	Software Requirements Specification
Software Engineering	[put an expanded version of your program name here (as appropriate) —TPLT]
TM	Theoretical Model

[Add any other abbreviations or acronyms that you add —TPLT]

1.4 Mathematical Notation

[This section is optional, but should be included for projects that make use of notation to convey mathematical information. For instance, if typographic conventions (like bold face font) are used to distinguish matrices, this should be stated here. If symbols are used to show mathematical operations, these should be summarized here. In some cases the easiest way to summarize the notation is to point to a text or other source that explains the notation. —TPLT]

[This section was added to the template because some students use very domain specific notation. This notation will not be readily understandable to people outside of your domain. It should be explained. —TPLT]

[This SRS template is based on Smith and Lai (2005); Smith et al. (2007); Smith and Koothoor (2016). It will get you started. You should not modify the section headings, without first discussing the change with the course instructor. Modification means you are not following the template, which loses some of the advantage of a template, especially standardization. Although the bits shown below do not include type information, you may need to add this information for your problem. If you are unsure, please can ask the instructor. —TPLT]

[Feel free to change the appearance of the report by modifying the LaTeX commands. —TPLT]

[This template document assumes that a single program is being documented. If you are documenting a family of models, you should start with a commonality analysis. A separate template is provided for this. For program families you should look at Smith (2006); Smith et al. (2017). Single family member programs are often programs based on a single physical model. General purpose tools are usually documented as a family. Families of physical models also come up. —TPLT]

[The SRS is not generally written, or read, sequentially. The SRS is a reference document. It is generally read in an ad hoc order, as the need arises. For writing an SRS, and for reading one for the first time, the suggested order of sections is:

- Goal Statement
- Instance Models
- Requirements
- Introduction
- Specific System Description

—TPLT]

[Guiding principles for the SRS document:

- Do not repeat the same information at the same abstraction level. If information is repeated, the repetition should be at a different abstraction level. For instance, there will be overlap between the scope section and the assumptions, but the scope section will not go into as much detail as the assumptions section.

—TPLT]

[The template description comments should be disabled before submitting this document for grading. —TPLT]

[You can borrow any wording from the text given in the template. It is part of the template, and not considered an instance of academic integrity. Of course, you need to cite the source of the template. —TPLT]

[When the documentation is done, it should be possible to trace back to the source of every piece of information. Some information will come from external sources, like terminology. Other information will be derived, like General Definitions. —TPLT]

[An SRS document should have the following qualities: unambiguous, consistent, complete, validatable, abstract and traceable. —TPLT]

[The overall goal of the SRS is that someone that meets the Characteristics of the Intended Reader (Section 2.3) can learn, understand and verify the captured domain knowledge. They should not have to trust the authors of the SRS on any statements. They should be able to independently verify/derive every statement made. —TPLT]

2 Introduction

A child's phonemic inventory (the sounds they are able to make) typically stabilizes by ages 4-6 [SPEECH SOUND DEVELOPMENT CHART], but this is not the case for many. In Ontario, roughly 1 in 10 children need extra help developing their speech and language skills [ONTARIO SLP FACTS]. Receiving proper treatment has proven to improve their self esteem, ability to socialize, and readiness for school [ONTARIO SLP FACTS]. The job of an SLP is to provide this treatment: they assess a child's speech, fluency, and articulation, and provide corresponding strategies to overcome their concerns.

The product "Pronounced Fish" is an assistive tool intended for SLPs to streamline their workflow, and offer complementary insights to assist with their consultations. This Software Requirements Specification (SRS) highlights all functionalities and expectations that guide the application's design. The current section describes the purpose of this document, project scope decisions, characteristics of the intended reader for the SRS, and a roadmap describing the topics covered by each section of the document.

2.1 Purpose of Document

The SRS will provide the formal requirements specification that any iteration of the product must adhere to. It compiles functional and non-functional requirements elicited from SLPs, children, and other relevant user groups that will guide the core development of this application. It will also outline some elements that are likely and unlikely to change through renditions of the product due to varying scope decisions, stakeholder needs, and user goals. A full list of goals and stakeholders can be found in the Problem Statement and Goals Document.

2.2 Scope of Requirements

Pediatric speech is very complicated because children are still learning how to coordinate their breath, lips, and brain to make certain sounds. This means it is normal for them to make mistakes around this age. By age 5, these features have generally stabilized so if a child is still making errors around this time, it suggests a need for speech therapy. Therefore, the product narrows down the target user space to children aged 4-6.

Speech errors can be narrowed down to one of four categories as per the SODA framework. This product focuses on addressing substitution errors: mistakes where children replace

consonant sounds with the wrong one. For instance, pronouncing the word "rabbit" as "wabbit" substitutes the "r" for a "w".

Another factor that complicates analysis of a child's speech is their accent. Accents are tuned to pronounce certain sounds in certain ways, causing significant deviations between youth from different geographical regions. Therefore, the user space has been further narrowed down to handle Monolingual North American speakers.

This means that the scope has been constrained to address substitution speech errors for monolingual, North American English speakers aged between 4-6.

2.3 Characteristics of Intended Reader

This document acts as the bridge between the actual application and the project developers. The reader is expected to have the equivalent software design understanding of a McMaster University Level 3 Software Engineering student. They must have a first year engineering level understanding of linear algebra and vector difference. This is important to understand the accuracy and comparisons of different speech sounds. A very basic understanding of linguistics terminology is also needed. They must know what a phoneme is, have an idea of some common speech sound errors (i.e. SODA), and understand the purpose of an SLP.

2.4 Organization of Document

The document is organized as per the following template [Smith and Lai (2005); Smith et al. (2007); Smith and Koothoor (2016)]:

- **Section 1:** Contains reference material starting with a table of units, symbols, abbreviations, and mathematical notation.
- **Section 2:** The current section introduces the problem, purpose of the SRS, scope of the document, and characteristics of the intended reader.
- **Section 3:** Provides a high level description of the system context including its inputs, outputs, and the interfaces between it and its environment. These system characteristics are tied to those of the intended user, and the system's constraints.
- **Section 4:** Highlights the overarching **problem statement and goals**. It then expands on any modelling decisions, scope decisions, and assumptions used to shape the solution space.
- **Section 5:** Outlines all functional and non-functional requirements the system must abide by, and the rationale for including them.
- **Section 6:** Describes any likely changes that are expected to be made in future iterations of the software. These changes will add or enhance preexisting requirements.
- **Section 7:** Describes any elements of the software that are unlikely to change.

- **Section 8:** Maps traceability matrices that connect related requirements, assumptions, and constraints.
- **Section 9:** Describes the order in which the requirements will be implemented during the development phase.
- **References:** Citations for any referenced materials within the document.
- **Appendix:** Contains a group reflection on the developmental process of this document.

3 General System Description

This section provides general information about the system. It identifies the interfaces between the system and its environment, describes the user characteristics and lists the system constraints. [This text can likely be borrowed verbatim. —TPLT]

[The purpose of this section is to provide general information about the system so the specific requirements in the next section will be easier to understand. The general system description section is designed to be changeable independent of changes to the functional requirements documented in the specific system description. The general system description provides a context for a family of related models. The general description can stay the same, while specific details are changed between family members. —TPLT]

3.1 System Context

[Your system context will include a figure that shows the abstract view of the software. Often in a scientific context, the program can be viewed abstractly following the design pattern of Inputs → Calculations → Outputs. The system context will therefore often follow this pattern. The user provides inputs, the system does the calculations, and then provides the outputs to the user. The figure should not show all of the inputs, just an abstract view of the main categories of inputs (like material properties, geometry, etc.). Likewise, the outputs should be presented from an abstract point of view. In some cases the diagram will show other external entities, besides the user. For instance, when the software product is a library, the user will be another software program, not an actual end user. If there are system constraints that the software must work with external libraries, these libraries can also be shown on the System Context diagram. They should only be named with a specific library name if this is required by the system constraint. —TPLT]

[For each of the entities in the system context diagram its responsibilities should be listed. Whenever possible the system should check for data quality, but for some cases the user will need to assume that responsibility. The list of responsibilities should be about the inputs and outputs only, and they should be abstract. Details should not be presented here. However, the information should not be so abstract as to just say “inputs” and “outputs”. A summarizing phrase can be used to characterize the inputs. For instance, saying “material



Figure 1: System Context

properties" provides some information, but it stays away from the detail of listing every required properties. —TPLT]

- User Responsibilities:
  - –
- Software Engineering Responsibilities:
  - Detect data type mismatch, such as a string of characters instead of a floating point number
  - –

[Identify in what context the software will typically be used. Is it for exploration? education? engineering work? scientific work?. Identify whether it will be used for mission-critical or safety-critical applications. —TPLT] [This additional context information is needed to determine how much effort should be devoted to the rationale section. If the application is safety-critical, the bar is higher. This is currently less structured, but analogous to, the idea to the Automotive Safety Integrity Levels (ASILs) that McSCert uses in their automotive hazard analyses. —TPLT]

3.2 User Characteristics

[This section summarizes the knowledge/skills expected of the user. Measuring usability, which is often a required non-function requirement, requires knowledge of a typical user. As mentioned above, the user is a different role from the "intended reader," as given in Section 2.3. As in Section 2.3, the user characteristics should be specific an unambiguous. For instance, "The end user of Software Engineering should have an understanding of undergraduate Level 1 Calculus and Physics." —TPLT]

3.3 System Constraints

[System constraints differ from other type of requirements because they limit the developers' options in the system design and they identify how the eventual system must fit into the world. This is the only place in the SRS where design decisions can be specified. That is,

the quality requirement for abstraction is relaxed here. However, system constraints should only be included if they are truly required. —TPLT]

4 Specific System Description

4.0.1 Scope Decisions

The following decisions are made to narrow down the scope of the project:

- **SD1**: The target demographic will be children aged 4-6.
  Rationale: A child's phonemic inventory (the sounds they are able to speak) typically stabilizes by ages 4-6 [SPEECH SOUND DEVELOPMENT CHART]. If a child has speech development issues at this point, they are referred to an SLP, making this a logical age range to choose. There is a lot more variation in speech at younger ages which may be normal, so younger age ranges are unrealistic to analyze for a capstone.

- **SD2**: The children will be monolingual speakers with a North-American accent.
  Rationale: Speech recognition tools struggle with recognizing mistakes in bilingual people or with accented speech [BIAS WITH ACCENTED SPEECH]. These speakers incorrectly get flagged as having speech disorders more often due to a lack of training data available for them [BIAS WITH ACCENTED SPEECH]. This means a general all-encompassing speech analysis model is unrealistic, and the project must focus on a space with sufficient training data to work from.

- **SD3**: The detected speech errors will be limited to consonant substitution errors.
  Rationale: There are four main types of speech errors as per the SODA framework, and addressing them all is a significant task. For this capstone, the main issues targeted are consonant substitution errors as they are the most common [COMMON SPEECH ERRORS] and are unaffected by dialects, accents, and ways of speaking. This makes them feasible to target.

- **SD4**: Supported devices will be limited to iPadOS, AndroidOS, Windows desktops, and Macs.
  Rationale: When eliciting requirements with SLPs, the majority of them stated to use iPads to supplement their sessions. It is unrealistic to develop a native application for each and every operating system within the capstone timeline, so the most common platforms will be picked.

4.0.2 Assumptions

- **A1: Single-word attempts.** Each attempt consists of a single spoken target word.

- **A2: Speaker population.** Each speaker is a monolingual North American English-speaking child aged 4–6.

- **A3: Expected phoneme sequence.** Each target word is drawn from a standardized word set and has a predefined expected phoneme sequence $T = (t_1, \ldots, t_n)$.
- **A4: Acceptable pronunciation variants.** Each target word has a finite, predefined set of acceptable phoneme variants V_i at each position i, where $t_i \in V_i$.
  - These sets are established in consultation with an SLP advisor to account for acceptable pronunciation variation.
- **A5: Substitution errors.** Each substitution error involves replacing exactly one expected phoneme with a different phoneme at the same position.
- **A6: Sequence length mismatch.** Any difference between the recognized phoneme sequence length and the expected length n is assumed to indicate at least one omission or addition.
- **A7: SLP supervision.** Each session is conducted under the direct supervision of an SLP.
- **A8: Unintentional pronunciation errors.** All pronunciation errors made by the speaker are assumed to be unintentional.
- **A9: Recording conditions.** Each recording contains speech from only one speaker and is captured in a quiet environment with minimal background noise.
- **A10: Calibrated phoneme confidence.** The recognizer's per-phoneme posterior probabilities are assumed to be calibrated, such that predicted confidence values correspond to observed frequencies of correct recognition.
- **A11: Session devices.** SLPs are assumed to conduct sessions using an iPad, Android tablet, or computer equipped with a microphone and a modern web browser.

4.0.3 Theoretical Models

- **TM1: Phoneme recognition.** $R(x) = (P, \mathbf{c})$, where x is the recorded audio, $P = (p_1, \ldots, p_m)$ is the recognized phoneme sequence, and $\mathbf{c} = (c_1, \ldots, c_m)$ are per-phoneme posteriors with $c_j \in [0, 1]$.
  - Abstract mapping from audio to phonemes with confidence.
- **TM2: Phoneme sequence comparison.** Given T (expected) and P (recognized) with $|P| = n$, pair t_i with p_i for $i = 1, \ldots, n$.
  - Positional comparison is sufficient for substitutions, which are the only errors in scope.

4.0.4 General Definitions

- **GD1: Substitution at position i.** $s_i = 1$ if $p_i \notin V_i$ and $(t_i \to p_i) \in L$, otherwise $s_i = 0$, where L is the predetermined list of substitution errors.

4.0.5 Data Definitions

- **DD1: Word position.** $\text{pos}(i)$ is *beginning* if $i = 1$; *end* if $i = n$; otherwise *middle*.
  - Used to report where in the word each substitution occurs.

5 Requirements

[The requirements refine the goal statement. They will make heavy use of references to the instance models. —TPLT]

This section provides the functional requirements, the business tasks that the software is expected to complete, and the nonfunctional requirements, the qualities that the software is expected to exhibit.

5.1 Functional Requirements

5.1.1 Audio Capture Requirements

- **FR-AC1:** The system shall capture the patient's spoken response as audio for each attempt.
  Rationale: Every analysis result is derived from the recording of an attempt.

- **FR-AC2:** The system shall use captured audio only to analyze the attempt it was captured for.
  Rationale: The system is scoped to data analysis only. Limiting the use of children's audio reduces privacy exposure under PHIPA.

- **FR-AC3:** The system shall flag to the SLP any recording that fails the usability limits (noise, clipping, duration).
  Rationale: Noisy or malformed audio is a main cause of misclassification, so the SLP must know when a result may be unreliable.

5.1.2 Session Management Requirements

- **FR-SM1:** The system shall allow the SLP to set up a session by selecting a patient, one or more target phonemes, and target items.
  Rationale: Target phonemes and items define what is practised and what the analysis compares against. Selectable phonemes are limited to those covered by the predetermined substitution list (FR-SA1).

- **FR-SM2:** The system shall allow the SLP to start, repeat, skip and end each attempt, and to end the session.
  Rationale: The SLP supervises the session and controls its pace.

- **FR-SM3:** The system shall allow the SLP to record, for each attempt, whether the target was produced independently, after a semantic cue, or after repetition.
  Rationale: The amount of support a patient needs is a clinical indicator of progress and is tracked across sessions.

- **FR-SM4:** The system shall allow the SLP to override the system's classification of any attempt.
  Rationale: The system's output is assistive and not diagnostic, so the SLP's clinical judgement takes precedence.

5.1.3 Speech Analysis Requirements

- **FR-SA1:** The system shall identify, from the captured audio of each attempt, substitutions from a predetermined list of common substitution errors, reporting the expected phoneme, the produced phoneme, and the word position.
  Rationale: Substitutions are the only error type in scope for the proof of concept, and accuracy is only validated for the listed errors. Word position supports the position-based visualization.

- **FR-SA2:** The system shall not identify a substitution when the produced phoneme matches any acceptable variant defined for the target item.
  Rationale: Accepted North American English pronunciation varies by dialect, and treating those variants as errors would produce false substitutions.

- **FR-SA3:** The system shall report a confidence value with each result and shall classify an attempt as "unclassifiable" when confidence is below the confidence threshold determined by the validation study.
  Rationale: The SLP can weigh uncertain results, and the system avoids forcing a classification on poor input, which reduces over-reliance on the tool.

5.1.4 Audio Capture Requirements

- **FR-AC1:** The system shall capture the patient's spoken response as audio for each attempt.
  Rationale: Every analysis result is derived from the recording of an attempt.

- **FR-AC2:** The system shall use captured audio only to analyze the attempt it was captured for.
  Rationale: The system is scoped to data analysis only. Limiting the use of children's audio reduces privacy exposure under PHIPA.

- **FR-AC3:** The system shall flag to the SLP any recording that fails the usability limits (noise, clipping, duration).
  Rationale: Noisy or malformed audio is a main cause of misclassification, so the SLP must know when a result may be unreliable.

5.1.5 Session Management Requirements

- **FR-SM1:** The system shall allow the SLP to set up a session by selecting a patient, one or more target phonemes, and target items.
  Rationale: Target phonemes and items define what is practised and what the analysis compares against. Selectable phonemes are limited to those covered by the predetermined substitution list (FR-SA1).

- **FR-SM2:** The system shall allow the SLP to start, repeat, skip and end each attempt, and to end the session.
  Rationale: The SLP supervises the session and controls its pace.

- **FR-SM3:** The system shall allow the SLP to record, for each attempt, whether the target was produced independently, after a semantic cue, or after repetition.
  Rationale: The amount of support a patient needs is a clinical indicator of progress and is tracked across sessions.

- **FR-SM4:** The system shall allow the SLP to override the system's classification of any attempt.
  Rationale: The system's output is assistive and not diagnostic, so the SLP's clinical judgement takes precedence.

5.1.6 Speech Analysis Requirements

- **FR-SA1:** The system shall identify, from the captured audio of each attempt, substitutions from a predetermined list of common substitution errors, reporting the expected phoneme, the produced phoneme, and the word position.
  Rationale: Substitutions are the only error type in scope for the proof of concept, and accuracy is only validated for the listed errors. Word position supports the position-based visualization.

- **FR-SA2:** The system shall not identify a substitution when the produced phoneme matches any acceptable variant defined for the target item.
  Rationale: Accepted North American English pronunciation varies by dialect, and treating those variants as errors would produce false substitutions.

- **FR-SA3:** The system shall report a confidence value with each result and shall classify an attempt as "unclassifiable" when confidence is below the confidence threshold determined by the validation study.

Rationale: The SLP can weigh uncertain results, and the system avoids forcing a classification on poor input, which reduces over-reliance on the tool.

[Every IM should map to at least one requirement, but not every requirement has to map to a corresponding IM. —TPLT]

5.2 Nonfunctional Requirements

[List your nonfunctional requirements. You may consider using a fit criterion to make them verifiable. —TPLT] [The goal is for the nonfunctional requirements to be unambiguous, abstract and verifiable. This isn't easy to show succinctly, so a good strategy may be to give a "high level" view of the requirement, but allow for the details to be covered in the Verification and Validation document. —TPLT] [An absolute requirement on a quality of the system is rarely needed. For instance, an accuracy of 0.0101 % is likely fine, even if the requirement is for 0.01 % accuracy. Therefore, the emphasis will often be more on describing now well the quality is achieved, through experimentation, and possibly theory, rather than meeting some bar that was defined a priori. —TPLT] [You do not need an entry for correctness in your NFRs. The purpose of the SRS is to record the requirements that need to be satisfied for correctness. Any statement of correctness would just be redundant. Rather than discuss correctness, you can characterize how far away from the correct (true) solution you are allowed to be. This is discussed under accuracy. —TPLT]

NFR1: **Accuracy**

NFR2: **Performance**

- Phonemic estimation results must be presented in less than 10 seconds for a short sentence. **Rationale:** This is because 10 seconds is the limit for sustaining a user's attention for in a human-computer dialogue before they will want to perform other tasks sof.

NFR3: **Usability**

- The SLP must be able to fully navigate the software within one hour of use. **Rationale:** We expect that it will take some time to become accustomed to the software, but we do not want to expect it to be a time sink.
- The SLP cannot delete critical data without a confirmation or the ability to backtrack within 72 hours. **Rationale:** This provides a safety net for the SLP to prevent accidental deletion of critical data.
- The time spent using the software tool during and outside of the session should be less than the time spent using equivalent manual tools. **Rationale:** This is inherently a tool that is meant for efficiency and any loss in efficiency is counterproductive.

- The child cannot access the rest of the system beyond the gamified experience. **Rationale:** This is another safety net to ensure that the child cannot access data without the supervision of the SLP.

NFR4: **Maintainability**

- The software should be modular to allow extenstion to interfaces for other professions (e.g. accent coach, language teachers). **Rationale:** We anticipate the value of extending the software, and a modular design facilitates this.
- The software should be modular to allow extenstion to other speech differences under SODA. **Rationale:** We anticipate the value of extending the software, and a modular design facilitates this.
- Build and deployment of a patch should take less than 30 minutes. **Rationale:** This is to ensure that the software is updated in a timely manner, ideally before the next session with a child for critical bug fixes.

NFR5: **Portability**

- Web-based application should work on Windows, MacOS, iOS, and Android OS. **Rationale:** Although this is a web-based application, basic OS compatibility still needs to be ensured on common platforms.
- Web-based application should work on Chrome, Firefox, Safari, and Edge browsers. **Rationale:** This covers some of the most commonly used browsers, including default ones such as Safari on MacOS and Edge on Windows.
- Web-based application should work on desktop, tablet, and laptop devices. **Rationale:** The majority of the SLPs will likely use one of these devices, with the most common being iPads (tablets) L. Turkstra.
- Other NFRs that might be discussed include verifiability, understandability and reusability.

6 Likely Changes

Below is a list of requirements that are likely to change. The provided rationale will explain whether this is due to a scope decision, stretch goal, or other factor.

- **LC1**: The speech analysis model shall identify omission, distortion and addition phonetic articulation errors as described in the SODA framework.
  Rationale: Due to scope requirements, speech error detection has been limited to just substitution errors for this project. Ideally, this will be changed to analyze all forms of errors.
  Related Assumption(s): A5, A6

- **LC2**: The speech analysis model will fairly accommodate accents originating from regions outside North America. Fair implies the model will recognize errors at a comparable accuracy (+/- 5% relative to the North American accent) for speakers of any accent.
  Rationale: Due to scope requirements, the child's accent has been limited to just Monolingual North American speakers. English speakers are very diverse, and there must be adequate support for users from all backgrounds.
  Related Assumption(s): A2

- **LC3**: The system shall provide an interface for an SLP to assign homework to their patients.
  Rationale: The first iteration of the system will provide an interface for an SLP-led session in the clinic. A stretch goal is to allow patients to complete homework at home, but this is not a core feature, so it is a likely change.
  Related Assumption(s): A7

- **LC4**: The system shall highlight errors for children of all age ranges based on the standardized speech sound development chart.
  Rationale: Due to scope requirements, the target child age has been restricted to ages 4-6. Speech errors may occur at any age, and even for adults. Therefore, more flexibility will be crucial in future iterations of the product.
  Related Assumption(s): A2

- **LC5**: The system will support being locally hosted on Windows and Mac operating systems. The system will have compatible mobile apps for iOS and Android OS. The system will be supported in Safari and Google Chrome browsers.
  Rationale: SLPs may conduct their sessions using iPad, tablet, or computer microphones, so all platforms must be supported. They also deal with confidential patient health information, so web-based solutions may not be trusted.
  Related Assumption(s): A11

7 Unlikely Changes

LC1: [Give the unlikely changes. The design can assume that the changes listed will not occur. —TPLT]

8 Traceability Matrices and Graphs

The purpose of the traceability matrices is to provide easy references on what has to be additionally modified if a certain component is changed. Every time a component is changed, the items in the column of that component that are marked with an "X" may have to be modified as well. Table 1 shows the dependencies of theoretical models, general definitions,

data definitions, and instance models with each other. Table 2 shows the dependencies of instance models, requirements, and data constraints on each other. Table 3 shows the dependencies of theoretical models, general definitions, data definitions, instance models, and likely changes on the assumptions.

[You will have to modify these tables for your problem. —TPLT]

[The traceability matrix is not generally symmetric. If GD1 uses A1, that means that GD1's derivation or presentation requires invocation of A1. A1 does not use GD1. A1 is "used by" GD1. —TPLT]

[The traceability matrix is challenging to maintain manually. Please do your best. In the future tools (like Drasil) will make this much easier. —TPLT]

	TM??	TM??	TM??	GD??	GD??	DD??	DD??	DD??	DD??	IM??	IM??	IM?
TM??												
TM??			X									
TM??												
GD??												
GD??	X											
DD??				X								
DD??				X								
DD??												
DD??								X				
IM??					X	X	X				X	
IM??					X		X		X	X		
IM??		X										
IM??		X	X				X	X	X		X	

Table 1: Traceability Matrix Showing the Connections Between Items of Different Sections

The purpose of the traceability graphs is also to provide easy references on what has to be additionally modified if a certain component is changed. The arrows in the graphs represent dependencies. The component at the tail of an arrow is depended on by the component at the head of that arrow. Therefore, if a component is changed, the components that it points to should also be changed. Figure **??** shows the dependencies of theoretical models, general definitions, data definitions, instance models, likely changes, and assumptions on each other. Figure **??** shows the dependencies of instance models, requirements, and data constraints on each other.

	IM??	IM??	IM??	IM??	??	R??	R??
IM??		X				X	X
IM??	X			X		X	X
IM??						X	X
IM??		X				X	X
R??							
R??						X	
R??					X		
R??	X	X				X	X
R??	X						
R??		X					
R??			X				
R??				X			
R??			X	X			
R??		X					
R??		X					

Table 2: Traceability Matrix Showing the Connections Between Requirements and Instance Models

9 Development Plan

Proof of Concept: For the proof of concept we will focus on implementing the phonemic analysis for substitution erros and some APIs to expose the data statistics to the rest of the system. We intend to do accuracy and performance testing in this phase to ensure the phonemic analysis meets those non-functional requirements. **Phase 1:** This is the phase that will be presented at the Capstone Expo. The interfaces should be complete for an SLP with a login, data visualization, lesson planning tools, progress tracking for the patients, and a game for the patients. The backend should have a modular design, and a database. The database will be real but will not be clinically ready as it will be a free Cloud service. Some basic security measures, such as two-factor authentication, will be put in place to protect patient data. The system should work in every selected browser, operating system, and device. **Phase 2:** Additional phonemic analysis will be implemented for other types of errors in SODA. The interface will be expanded to support data analysis and progress tracking for these errors as well as specialized games. **Phase 3:** Security, privacy, and PHIPPA/medical requirements will be met in this phase. This is through proper agreements with Cloud services or the implementation of locally hosted services. Additional security measures will be implemented to ensure cybersecurity and the protection of malicious actors accessing sensitive data. This is the phase that ensures that the software is fully ready for

	A??	A??	A??	A??	A??	A??	A??	A??	A??	A??	A??	A??	A??	A??	A??	A??	A??	A??	A??
TM??	X																		
TM??																			
TM??																			
GD??		X																	
GD??			X	X	X	X													
DD??							X	X	X										
DD??			X	X						X									
DD??																			
DD??																			
IM??											X	X		X	X	X			X
IM??												X	X			X	X	X	
IM??														X					X
IM??													X					X	
LC??				X															
LC??								X											
LC??									X										
LC??											X								
LC??												X							
LC??															X				

Table 3: Traceability Matrix Showing the Connections Between Assumptions and Other Items

clinical deployment and use. **Phase 4 and on:** This is the phase where the software will be further into a line of software products for various professions and applications. Additional phonemic analysis may be required for this to detect speech differences that do not fall under SODA such as prosody.

[This section is optional. It is used to explain the plan for developing the software. In particular, this section gives a list of the order in which the requirements will be implemented. In the context of a course this is where you can indicate which requirements will be implemented as part of the course, and which will be "faked" as future work. This section can be organized as a prioritized list of requirements, or it could should the requirements that will be implemented for "phase 1", "phase 2", etc. —TPLT]

10 Values of Auxiliary Constants

[Show the values of the symbolic parameters introduced in the report. —TPLT]

[The definition of the requirements will likely call for SYMBOLIC_CONSTANTS. Their values are defined in this section for easy maintenance. —TPLT]

[The value of FRACTION, for the Maintainability NFR would be given here. —TPLT]

References

D. Pape L. Turkstra. Initial meeting with supervisors. In-person conversation.

W. Spencer Smith. Systematic development of requirements documentation for general purpose scientific computing software. In *Proceedings of the 14th IEEE International Requirements Engineering Conference, RE 2006*, pages 209–218, Minneapolis / St. Paul, Minnesota, 2006. URL http://www.ifi.unizh.ch/req/events/RE06/.

W. Spencer Smith and Nirmitha Koothoor. A document-driven method for certifying scientific computing software for use in nuclear safety analysis. *Nuclear Engineering and Technology*, 48(2):404–418, April 2016. ISSN 1738-5733. doi: http://dx.doi.org/10.1016/j.net.2015.11.008. URL http://www.sciencedirect.com/science/article/pii/S1738573315002582.

W. Spencer Smith and Lei Lai. A new requirements template for scientific computing. In J. Ralyté, P. Ȧgerfalk, and N. Kraiem, editors, *Proceedings of the First International Workshop on Situational Requirements Engineering Processes – Methods, Techniques and Tools to Support Situation-Specific Requirements Engineering Processes, SREP'05*, pages 107–121, Paris, France, 2005. In conjunction with 13th IEEE International Requirements Engineering Conference.

W. Spencer Smith, Lei Lai, and Ridha Khedri. Requirements analysis for engineering computation: A systematic approach for improving software reliability. *Reliable Computing, Special Issue on Reliable Engineering Computation*, 13(1):83–107, February 2007.

W. Spencer Smith, John McCutchan, and Jacques Carette. Commonality analysis for a family of material models. Technical Report CAS-17-01-SS, McMaster University, Department of Computing and Software, 2017.

[The following is not part of the template, just some things to consider when filing in the template. —TPLT]
[Grammar, flow and LaTeXadvice:

- For Mac users `*.DS_Store` should be in `.gitignore`
- LaTeX and formatting rules
  - Variables are italic, everything else not, includes subscripts (link to document)
    - Conventions
    - Watch out for implied multiplication
  - Use BibTeX
  - Use cross-referencing
- Grammar and writing rules
  - Acronyms expanded on first usage (not just in table of acronyms)
  - "In order to" should be "to"

—TPLT]
[Advice on using the template:

- Difference between physical and software constraints
- Properties of a correct solution means *additional* properties, not a restating of the requirements (may be "not applicable" for your problem). If you have a table of output constraints, then these are properties of a correct solution.
- Assumptions have to be invoked somewhere
- "Referenced by" implies that there is an explicit reference
- Think of traceability matrix, list of assumption invocations and list of reference by fields as automatically generatable
- If you say the format of the output (plot, table etc), then your requirement could be more abstract

—TPLT]

Appendix — Reflection

[Not required for CAS 741 —SS]

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. How many of your requirements were inspired by speaking to your client(s) or their proxies (e.g. your peers, stakeholders, potential users)?

4. Which of the courses you have taken, or are currently taking, will help your team to be successful with your capstone project.

5. What knowledge and skills will the team collectively need to acquire to successfully complete this capstone project? Examples of possible knowledge to acquire include domain specific knowledge from the domain of your application, or software engineering knowledge, mechatronics knowledge or computer science knowledge. Skills may be related to technology, or writing, or presentation, or team management, etc. You should look to identify at least one item for each team member.

6. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?

References

D. Pape L. Turkstra. Initial meeting with supervisors. In-person conversation.

W. Spencer Smith. Systematic development of requirements documentation for general purpose scientific computing software. In *Proceedings of the 14th IEEE International Requirements Engineering Conference, RE 2006*, pages 209–218, Minneapolis / St. Paul, Minnesota, 2006. URL http://www.ifi.unizh.ch/req/events/RE06/.

W. Spencer Smith and Nirmitha Koothoor. A document-driven method for certifying scientific computing software for use in nuclear safety analysis. *Nuclear Engineering and Technology*, 48(2):404–418, April 2016. ISSN 1738-5733. doi: http://dx.doi.org/10.1016/j.net.2015.11.008. URL http://www.sciencedirect.com/science/article/pii/S1738573315002582.

W. Spencer Smith and Lei Lai. A new requirements template for scientific computing. In J. Ralyté, P. Ågerfalk, and N. Kraiem, editors, *Proceedings of the First International Workshop on Situational Requirements Engineering Processes – Methods, Techniques and Tools to Support Situation-Specific Requirements Engineering Processes, SREP'05*, pages 107–121, Paris, France, 2005. In conjunction with 13th IEEE International Requirements Engineering Conference.

W. Spencer Smith, Lei Lai, and Ridha Khedri. Requirements analysis for engineering computation: A systematic approach for improving software reliability. *Reliable Computing, Special Issue on Reliable Engineering Computation*, 13(1):83–107, February 2007.

W. Spencer Smith, John McCutchan, and Jacques Carette. Commonality analysis for a family of material models. Technical Report CAS-17-01-SS, McMaster University, Department of Computing and Software, 2017.